

Section 3 -- Deploying & Implementing (~25% of exam)

GCP Study Notes

Study Notes April 2026

Compute Engine

```
# Create VM
gcloud compute instances create VM --zone=ZONE --machine-type=TYPE \
  --image-family=IMG --image-project=PROJ --boot-disk-type=pd-ssd \
  --tags=http-server --no-address --service-account=SA

# Create + attach disk
gcloud compute disks create DISK --size=200GB --type=pd-ssd --zone=ZONE
gcloud compute instances attach-disk VM --disk=DISK --zone=ZONE

# Instance template (immutable!)
gcloud compute instance-templates create TPL --machine-type=TYPE --image-family=IMG ...

# MIG
gcloud compute instance-groups managed create MIG --template=TPL --size=3 --zone=ZONE
# Regional MIG (HA)
gcloud compute instance-groups managed create MIG --template=TPL --size=6 --region=REGION

# Autoscaling
gcloud compute instance-groups managed set-autoscaling MIG --zone=ZONE \
  --min-num-replicas=2 --max-num-replicas=10 --target-cpu-utilization=0.70 --cool-down-per

# Rolling update (zero downtime)
gcloud compute instance-groups managed rolling-action start-update MIG \
  --version=template=NEW_TPL --zone=ZONE --max-surge=3 --max-unavailable=0

# Canary update
... --canary-version=template=NEW_TPL,target-size=20%

# SSH via IAP (no external IP)
gcloud compute ssh VM --zone=ZONE --tunnel-through-iap
```

Disk types: pd-standard (cheap) < pd-balanced (default) < pd-ssd < pd-extreme

OS Login: `--metadata enable-oslogin=TRUE` – uses IAM, supports 2FA – `roles/compute.osLogin`
 = non-root | `roles/compute.osAdminLogin` = sudo

Maintenance Policy	When
MIGRATE (default)	Production VMs
TERMINATE	GPU VMs, Spot VMs

- :warning: GPU VMs **must** use `--maintenance-policy=TERMINATE`
- :warning: Instance templates are **immutable** – create a new one to change config
- :warning: Boot disk auto-deletes with VM; additional disks do NOT
- :warning: Disks can only be **increased** in size, never decreased
- :warning: IAP SSH requires firewall rule allowing `35.235.240.0/20` on port 22

GKE

```
# Get kubectl credentials
gcloud container clusters get-credentials CLUSTER --zone=ZONE

# Standard cluster
gcloud container clusters create CL --zone=ZONE --num-nodes=3 --machine-type=e2-standard-4
--enable-autoscaling --min-nodes=2 --max-nodes=10 --release-channel=regular

# Autopilot cluster (Google manages nodes)
gcloud container clusters create-auto CL --region=REGION

# Private cluster
gcloud container clusters create CL --zone=ZONE \
--enable-private-nodes --master-ipv4-cidr=172.16.0.0/28 \
--enable-ip-alias --enable-master-authorized-networks
```

Key kubectl:

```
kubectl apply -f FILE.yaml
kubectl create deployment APP --image=IMAGE --replicas=3
kubectl expose deployment APP --type=LoadBalancer --port=80 --target-port=8080
kubectl scale deployment APP --replicas=5
kubectl set image deployment/APP CONTAINER=IMAGE:v2
kubectl rollout undo deployment/APP
kubectl logs POD
kubectl exec -it POD -- /bin/bash
```

Service types: ClusterIP (internal) | NodePort (dev) | LoadBalancer (external) | ExternalName

Workload Identity (recommended for pod -> GCP API access):

```
gcloud container clusters update CL --zone=ZONE --workload-pool=PROJECT.svc.id.goog
# Then: GCP SA -> iam.workloadIdentityUser -> K8s SA annotation
```

Standard vs Autopilot	Standard	Autopilot
Node mgmt	You	Google
Pricing	Per node VM	Per pod resources
Best for	GPU, full control	Most workloads, less ops

Release Channel | Rapid (newest) | Regular (default) | Stable (safest) | Extended (patches only) |

- :warning: Regional cluster `--region` = 3 control planes (HA); `--zone` = single SPOF
- :warning: Regional cluster: `--num-nodes=2` creates **2 per zone = 6 total**
- :warning: Private cluster nodes need **Cloud NAT** for outbound internet
- :warning: `get-credentials` updates `~/.kube/config` – must run before kubectl works

Cloud Run & Cloud Functions

```
# Cloud Run deploy
gcloud run deploy SVC --image=IMAGE --region=REGION --allow-unauthenticated \
  --port=8080 --memory=512Mi --cpu=1 --min-instances=0 --max-instances=100 \
  --set-env-vars="K=V" --service-account=SA --vpc-connector=CONN

# Cloud Run from source
gcloud run deploy SVC --source=. --region=REGION

# Traffic split (canary)
gcloud run services update-traffic SVC --region=REGION \
  --to-revisions=OLD_REV=90,NEW_REV=10

# Cloud Functions Gen 2 -- HTTP
gcloud functions deploy FN --gen2 --region=REGION --runtime=python312 \
  --trigger-http --allow-unauthenticated --entry-point=FUNC --source=.

# Cloud Functions Gen 2 -- Pub/Sub
gcloud functions deploy FN --gen2 --runtime=python312 --trigger-topic=TOPIC --entry-point=

# Cloud Functions Gen 2 -- GCS
gcloud functions deploy FN --gen2 --runtime=python312 \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=BUCKET" --entry-point=FUNC
```

Cloud Run ingress: all (default) | internal | internal-and-cloud-load-balancing

Gen 1 vs Gen 2	Gen 1	Gen 2
Timeout	9 min	60 min
Memory	8 GB	32 GB
Concurrency	1 req/instance	up to 1000
Traffic split	No	Yes
Built on	Custom	Cloud Run

When to use what:

Cloud Run**Cloud Functions**

Multiple endpoints/routes

Single-purpose function

Custom Dockerfile

Simple code, supported runtime

WebSocket/gRPC

Quick event processing

Traffic splitting needed

Glue code between services

- :warning: Gen 2 is **recommended** – always use `--gen2`
- :warning: Cloud Run default port is **8080**, not 80
- :warning: Each deploy creates an **immutable revision**
- :warning: `--allow-unauthenticated` = public; default is **authenticated**
- :warning: VPC access requires a **Serverless VPC Access connector**

Data Solutions

```
# Cloud SQL
gcloud sql instances create INST --database-version=MYSQL_8_0 \
  --tier=db-n1-standard-4 --region=REGION --availability-type=REGIONAL \
  --storage-type=SSD --storage-auto-increase --backup-start-time=02:00
gcloud sql databases create DB --instance=INST
gcloud sql users create USER --instance=INST --password=PW

# Firestore
gcloud firestore databases create --location=LOC --type=firestore-native

# BigQuery
bq mk --dataset --location=US PROJECT:dataset
bq mk --table PROJECT:dataset.table col1:STRING,col2:FLOAT
bq load --source_format=CSV --autodetect PROJECT:dataset.table gs://BUCKET/*.csv

# Spanner
gcloud spanner instances create INST --config=regional-us-central1 --processing-units=1000
gcloud spanner databases create DB --instance=INST

# Pub/Sub
gcloud pubsub topics create TOPIC
gcloud pubsub subscriptions create SUB --topic=TOPIC --ack-deadline=60
gcloud pubsub subscriptions create SUB --topic=TOPIC --push-endpoint=URL # push
gcloud pubsub subscriptions create SUB --topic=TOPIC --bigquery-table=PROJ:ds.tbl # BQ di

# AlloyDB
gcloud alloydb clusters create CL --region=REGION --password=PW --network=VPC
gcloud alloydb instances create INST --cluster=CL --region=REGION --instance-type=PRIMARY

# Cloud Storage
gcloud storage buckets create gs://BUCKET --location=LOC --default-storage-class=STANDARD
  --uniform-bucket-level-access

# Data loading
gcloud storage cp -r ./data/ gs://BUCKET/
bq load --source_format=CSV --autodetect PROJECT:ds.tbl gs://BUCKET/*.csv
gcloud sql import csv INST gs://BUCKET/data.csv --database=DB --table=TBL
gcloud dataflow jobs run JOB --gcs-location=gs://TEMPLATE --region=REGION --parameters=...
```

Transfer options: gsutil/gcloud (GBs) | bq load (GBs-TBs) | Storage Transfer Service (TBs-PBs, cross-cloud) | Transfer Appliance (PBs, offline) | Dataflow (transform+load)

Pub/Sub sub types: Pull (subscriber requests) | Push (HTTP delivery) | BigQuery (direct write) | Cloud Storage

Spanner: 1 node = 1000 processing units; min = 100 PU

- :warning: Firestore mode (Native vs Datastore) **cannot be changed** after creation
- :warning: Cloud SQL `REGIONAL` = HA with auto-failover; `ZONAL` = no failover
- :warning: Cloud SQL Auth Proxy is the **recommended** connection method
- :warning: `--enable-bin-log` required for MySQL point-in-time recovery
- :warning: Dataflow: `drain` = process remaining then stop; `cancel` = stop immediately

Networking

```
# Custom VPC
gcloud compute networks create VPC --subnet-mode=custom

# Subnet
gcloud compute networks subnets create SUB --network=VPC --region=REGION --range=10.0.1.0/
# With secondary ranges for GKE
... --secondary-range=pods=10.4.0.0/14,services=10.8.0.0/20
# Enable Private Google Access
gcloud compute networks subnets update SUB --region=REGION --enable-private-ip-google-acce

# Firewall rules
gcloud compute firewall-rules create RULE --network=VPC --allow=tcp:80 \
  --source-ranges=0.0.0.0/0 --target-tags=http-server --priority=1000 --direction=INGRESS
# IAP rule
... --allow=tcp:22 --source-ranges=35.235.240.0/20

# Shared VPC
gcloud compute shared-vpc enable HOST_PROJECT
gcloud compute shared-vpc associated-projects add SVC_PROJECT --host-project=HOST_PROJECT

# VPC Peering (BOTH sides required)
gcloud compute networks peerings create PEER --network=VPC_A --peer-network=VPC_B --peer-p

# HA VPN
gcloud compute vpn-gateways create GW --network=VPC --region=REGION
gcloud compute routers create RTR --network=VPC --region=REGION --asn=65001
gcloud compute vpn-tunnels create TUN --vpn-gateway=GW --peer-external-gateway=PEER \
  --ike-version=2 --shared-secret=SECRET --router=RTR --interface=0
```

Firewall priority: 0 (highest) to 65535 (lowest). Rules are **stateful**.

Targets: Network tags (simple) | Service accounts (more secure, recommended for prod) | All (no target)

VPC Peering**Shared VPC**

Connects 2 separate VPCs	One VPC shared across projects
Non-transitive	Centralized network admin
Both sides must configure	Host + service projects

Cloud VPN**Dedicated Interconnect****Partner Interconnect**

Encrypted (IPsec)	Not encrypted	Not encrypted
Up to 3 Gbps/tunnel	10-200 Gbps	50 Mbps-50 Gbps
Hours to set up	Weeks	Days
99.99% (HA VPN)	99.9-99.99%	99.9-99.99%

- :warning: Auto-mode to custom-mode conversion is **one-way, irreversible**
- :warning: VPC peering is **non-transitive** (A<->B + B<->C does NOT give A<->C)
- :warning: VPC peering requires **no overlapping IP ranges**
- :warning: Hierarchical firewall policies (org/folder) are evaluated **before** VPC rules
- :warning: Private Google Access is needed for internal-only VMs to reach Google APIs
- :warning: HA VPN requires **2+ tunnels** for 99.99% SLA; uses **BGP** (dynamic routing)

Infrastructure as Code

Terraform workflow: `init` -> `plan` -> `apply` -> `destroy`

```

terraform init          # download providers, configure backend
terraform plan          # dry run
terraform apply         # create/update resources
terraform destroy       # delete all
terraform import RES ID # import existing resource into state
terraform state list    # list managed resources
terraform state rm RES  # remove from state without deleting

```

State backend: Local (dev only) | **GCS (production)** – enable versioning + locking

Tool	Manages	Language	Requires	Multi-cloud
Terraform	Any cloud resource	HCL	CLI	Yes
CFT	GCP (Terraform modules)	HCL	CLI	No
Config Connector	GCP as K8s CRDs	YAML	GKE cluster	No
Helm	K8s apps	YAML+Go tpl	K8s cluster	N/A

Cloud Foundation Toolkit (CFT): Google-built, opinionated Terraform modules (project-factory, network, GKE, SQL, IAM)

Config Connector: Manage GCP resources via `kubectl apply` – continuous reconciliation, fits GitOps (ArgoCD/Flux)

Helm: K8s package manager. Chart = bundle of templates. `helm install`, `helm upgrade`, `helm rollback`

- :warning: Terraform state **never store locally in production** – use GCS backend
- :warning: Config Connector requires a **GKE cluster** to run
- :warning: If you delete a Terraform-managed resource manually, next `apply` **recreates** it
- :warning: Terraform state file contains **sensitive data** – secure the GCS bucket
- :warning: CFT modules are **Terraform modules**, not a separate tool